

Unidad 2: Distintos modelos NoSQL

Clase 5 – Modelos de grafos y análisis de relaciones complejas

Objetivo

Comprender el modelo de grafos y utilizar consultas básicas en Cypher para analizar relaciones complejas entre entidades, aplicando este enfoque a escenarios de detección de vínculos ocultos, colusión y fraude.

Tema

Modelo de grafos.

Cypher – Bases de datos de grafos

- Nodos, relaciones y propiedades
 - Consultas de patrones
 - Casos de uso: colusión, fraude, relaciones ocultas
- Ejercitación conceptual y práctica básica.

Durante muchos años, la mayor parte de los sistemas se construyó sobre bases relacionales. Ese enfoque sigue siendo muy valioso, sobre todo cuando predominan transacciones estructuradas y reglas de integridad bien definidas. Sin embargo, hay problemas donde el foco principal no está solo en los datos aislados sino en las conexiones entre ellos. En esos casos, el modelo de grafos aporta una ventaja importante porque permite navegar conjuntos altamente conectados de manera eficiente. Por eso hoy se usa en motores de recomendación, detección de fraude, knowledge graphs, descubrimiento de fármacos y seguridad de redes.

En auditoría, prevención de fraude y analítica investigativa, muchas señales relevantes no aparecen al mirar un registro por separado. Aparecen cuando se conectan personas, empresas, cuentas, dispositivos, domicilios, movimientos y eventos. Ese tipo de análisis relacional es justamente el terreno natural de los grafos. Un ejemplo actual es el uso de resolución de entidades y análisis de grafos para detectar fraude en pagos con tarjeta no presente, relacionando clientes, transacciones y patrones sospechosos, incluso mediante técnicas como community detection para descubrir conexiones indirectas.

Qué es el modelo de grafos

Un grafo es una representación lógica de un conjunto de nodos donde algunos pares están conectados mediante relaciones. En el modelo de grafo de propiedades, los nodos representan entidades del dominio, las relaciones representan conexiones entre esas entidades, y tanto nodos como relaciones pueden almacenar propiedades en

forma de pares clave–valor. Además, los nodos se clasifican mediante labels y las relaciones mediante tipos.

Esto implica un cambio de mirada respecto del modelo relacional. En una base relacional, las relaciones entre entidades suelen reconstruirse a partir de claves y JOINS. En una base de grafos, la conexión se modela de forma explícita. La pregunta deja de ser solamente “qué datos tiene cada tabla” y pasa a ser también “cómo se vinculan las entidades y qué patrones de conexión aparecen”. Esa diferencia conceptual es central para entender por qué los grafos resultan tan potentes en investigaciones complejas.

Componentes fundamentales del modelo

Nodos

Un nodo representa una entidad u objeto discreto del dominio. Puede ser una persona, una empresa, una cuenta bancaria, un proveedor, un dispositivo, una dirección IP, un domicilio o incluso un evento. Los nodos pueden tener propiedades y labels. Una label marca al nodo como miembro de un subconjunto nombrado e indexado, y un nodo puede tener cero o más labels.

Ejemplos de nodos en un caso de fraude:

Persona, Empresa, Cuenta, Dispositivo, Domicilio, Transacción.

Relaciones

Una relación representa una conexión entre nodos. Conecta un nodo origen con un nodo destino, puede tener propiedades y se clasifica por tipo. En Neo4j, las relaciones tienen dirección, aunque en consulta se puede decidir ignorarla cuando no resulta relevante.

Ejemplos de relaciones:

SOCIO_DE, USA_CUENTA, OPERA_DESDE, TRANSFIERE_A, USA_DISPOSITIVO, AUTORIZA, COMPARTE_DOMICILIO_CON.

Propiedades

Las propiedades son pares clave–valor que guardan información sobre nodos o relaciones. Por ejemplo, una persona puede tener nombre, dni y riesgo; una relación TRANSFIERE_A puede tener fecha, monto y canal.

Labels y tipos

Conviene no confundirlos. La label se aplica al nodo y ayuda a clasificar entidades, por ejemplo :Persona o :Empresa. El tipo se aplica a la relación y expresa el significado del vínculo, por ejemplo :SOCIO_DE o :USA_CUENTA.

Dirección

En grafos, la dirección no es un detalle estético. Forma parte del significado del vínculo. No es lo mismo `(:Persona)-[:AUTORIZA]->(:Pago)` que `(:Pago)-[:AUTORIZA]->(:Persona)`. Aunque algunas consultas pueden escribirse sin importar la dirección, el modelado correcto debe respetar el sentido del negocio. Neo4j indica esa dirección con flechas como `-->` o `<--`, y también permite patrones sin dirección cuando solo interesa saber si existe conexión.

Qué aporta un grafo que otros modelos resuelven con más dificultad

El punto fuerte del grafo aparece cuando interesa recorrer cadenas de vínculos. No solo saber si A se relaciona con B, sino si A se conecta con C a través de B, o si varias entidades aparentemente independientes comparten una misma cuenta, un mismo dispositivo o un mismo domicilio.

En ese sentido, el grafo es especialmente útil cuando el problema se formula como una red. Esto ocurre en casos de colusión entre proveedores, fraude con cuentas compartidas, ocultamiento de beneficiarios finales, movimientos entre empresas vinculadas, uso de money mules o redes de dispositivos asociados a múltiples usuarios. Europol viene señalando que las redes criminales se sostienen a través de corrupción y lavado, generando un sistema financiero oculto; además, identifica a los money mules como el vínculo visible entre el mundo online y el offline en el lavado asociado al cibercrimen. FATF, por su parte, destaca que compañías pantalla, estructuras societarias complejas y directores nominales pueden utilizarse para ocultar a los verdaderos beneficiarios.

Cuándo conviene pensar en grafos

Conviene pensar en un grafo cuando:

- el problema depende fuertemente de las relaciones entre entidades;
- interesa descubrir conexiones indirectas;
- hay que detectar comunidades, cadenas o redes;
- los JOINS del modelo relacional empiezan a multiplicarse y a volverse difíciles de explicar;
- el análisis requiere pasar de “registro por registro” a “red por red”.

No significa que el modelo de grafos reemplace a todos los demás. Significa que resulta especialmente adecuado cuando el vínculo entre entidades es tan importante como los datos de cada entidad. Por eso no debe verse como un modelo exótico, sino como una alternativa especializada para problemas altamente conectados.

Introducción a Cypher

Cypher es el lenguaje declarativo de Neo4j. “Declarativo” significa que uno describe el patrón que desea encontrar o crear, en lugar de programar paso a paso cómo recorrer la estructura. Esa idea resulta muy didáctica porque la consulta se parece bastante al dibujo lógico del grafo.

Su sintaxis básica gira alrededor de tres formas:

- Los nodos se escriben entre paréntesis: (p)
- Las relaciones se escriben entre corchetes: [r]
- La dirección se expresa con flechas: -> o <-

Además:

- :Label identifica la clase del nodo.
- :TIPO_RELACION identifica el tipo de relación.
- {} contiene propiedades.
- Las variables permiten reutilizar nodos y relaciones dentro de la consulta.

Ejemplos:

```
(p:Persona)
(e:Empresa {nombre: 'Alpha SA'})
(p)-[:SOCIO_DE]->(e)
```

Esa última expresión significa: un nodo p de tipo Persona se relaciona con un nodo e de tipo Empresa mediante una relación SOCIO_DE.

Operaciones básicas que los alumnos deben aprender primero

CREATE

CREATE permite crear nodos y relaciones. Cada nodo puede recibir labels y propiedades. Las relaciones también pueden tener propiedades, pero deben tener exactamente un tipo y una dirección.

Ejemplo:

```
CREATE (p:Persona {nombre: 'Laura', dni: '30111222'})
```

Ejemplo con relación:

```
CREATE (p:Persona {nombre: 'Laura'})-[:SOCIO_DE]->(e:Empresa {nombre: 'Nova SRL'})
```

MATCH

MATCH se usa para buscar patrones dentro del grafo. Puede buscar nodos, relaciones o estructuras más complejas. Es la cláusula central para leer información.

Ejemplo:

```
MATCH (p:Persona)
RETURN p
```

Ejemplo con relación:

```
MATCH (p:Persona)-[:SOCIO_DE]->(e:Empresa)
RETURN p, e
```

WHERE

WHERE agrega restricciones al patrón que se está buscando. No debe pensarse solo como un filtro posterior, sino como parte del patrón de búsqueda.

Ejemplo:

```
MATCH (p:Persona)-[:SOCIO_DE]->(e:Empresa)
WHERE e.nombre = 'Nova SRL'
RETURN p.nombre
```

RETURN

RETURN define qué parte del patrón aparecerá en el resultado: nodos, relaciones, propiedades o expresiones derivadas.

Ejemplo:

```
MATCH (p:Persona)-[:SOCIO_DE]->(e:Empresa)
RETURN p.nombre, e.nombre
```

Cómo leer visualmente una consulta Cypher

Una de las ventajas didácticas de Cypher es que se puede “leer” casi como si uno mirara un diagrama.

```
MATCH (p:Persona)-[:SOCIO_DE]->(e:Empresa)
RETURN p.nombre, e.nombre
```

La lectura natural sería: “buscar personas que sean socias de empresas y devolver el nombre de la persona y el nombre de la empresa”.

Otro ejemplo:

```
MATCH (e:Empresa)-[:USA_CUENTA]->(c:Cuenta)<-[:USA_CUENTA]-(otra:Empresa)
RETURN e.nombre, otra.nombre, c.cbu
```

La lectura sería: “buscar empresas que usan una misma cuenta y devolver ambas empresas junto con el CBU compartido”.

Ese modo de lectura ayuda mucho a los alumnos porque reduce la distancia entre el modelo conceptual y la consulta.

Qué conceptos deben quedar claros después de estas primeras consultas

Primero, que Cypher no consulta “tablas unidas”, sino patrones de conexión. Segundo, que una relación tiene valor analítico propio y no debe pensarse solo como un puente técnico. Tercero, que el resultado relevante muchas veces no es un dato individual sino una estructura: una cadena, una coincidencia, una convergencia o una comunidad de nodos vinculados.

También debe quedar claro que no todo atributo repetido conviene transformarlo automáticamente en nodo. Un valor se vuelve nodo cuando tiene sentido analítico como entidad conectable. Por ejemplo, una cuenta, un domicilio o un dispositivo sí suelen merecer su propio nodo si interesa detectar reutilización o coincidencias. En cambio, un color o una descripción breve muchas veces alcanza con dejarlo como propiedad.

Errores frecuentes al empezar con grafos

Un error común es modelar casi todo como propiedad y perder conexiones útiles. Si una dirección solo está escrita como texto dentro de cada empresa, será más difícil detectar que varias empresas comparten exactamente el mismo domicilio. En cambio, si el domicilio es un nodo, la coincidencia aparece naturalmente.

Otro error frecuente es usar nombres de relaciones poco expresivos. En grafos conviene que el tipo de relación tenga significado de negocio. SOCIO_DE, AUTORIZA, TRANSFIERE_A o OPERA_DESDE comunican mucho mejor que nombres genéricos.

También es habitual confundir label con propiedad. :Empresa no es lo mismo que {tipo: 'empresa'}. La label clasifica al nodo dentro del modelo; la propiedad almacena información sobre él. Neo4j distingue claramente labels, tipos de relación y propiedades.

Relación entre grafos, fraude y colusión

Los grafos no son solo una forma distinta de guardar datos. Son una forma distinta de pensar el problema. En un análisis de colusión, por ejemplo, no basta con revisar ofertas aisladas. Puede ser necesario ver si distintos proveedores comparten accionistas, domicilios, apoderados, cuentas o teléfonos. En un análisis de fraude digital, puede ser necesario ver si múltiples usuarios operan desde el mismo dispositivo, la misma IP o la misma cuenta intermediaria. En lavado o beneficiario final, puede ser necesario atravesar capas societarias hasta encontrar quién controla realmente una estructura. FATF remarca justamente que las compañías pantalla, las estructuras societarias complejas y los intermediarios pueden usarse para ocultar la propiedad real, mientras que Europol advierte que el crimen organizado se apoya en redes financieras ocultas y en mecanismos intermediarios para mover fondos.

Por eso, el grafo también debe entenderse como una herramienta de análisis relacional profundo. No reemplaza el pensamiento crítico del auditor o del analista,

pero le da una estructura mucho más poderosa para descubrir patrones que en otros modelos quedarían escondidos.

El modelo relacional sigue siendo excelente para consistencia transaccional, normalización y explotación tabular. El modelo de grafos sobresale cuando la pregunta central depende de conexiones y recorridos. No es que uno sea “mejor” en términos absolutos; cada uno resuelve mejor ciertos problemas. Pero en análisis de relaciones complejas, el grafo ofrece una representación mucho más natural del dominio y una forma de consulta mucho más cercana a la lógica del caso.

El modelo de grafos resulta especialmente potente cuando el problema no puede entenderse observando entidades aisladas, sino la red de vínculos entre ellas. Nodos, relaciones y propiedades forman la base del modelo. Cypher permite expresar patrones de manera legible y directa. Y su valor aparece con fuerza en escenarios como fraude, colusión, beneficiario final oculto, movimientos intermediados y redes de comportamiento compartido. Desde el punto de vista pedagógico, esta clase marca el paso desde “almacenar datos” hacia “interpretar conexiones”.

Actividad práctica — Instalación de Neo4j y primeros pasos con grafos

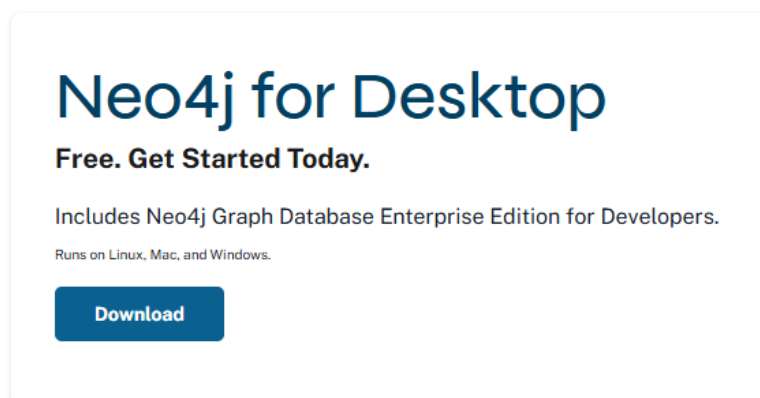
El objetivo de esta actividad es instalar Neo4j Desktop, crear una base de datos local, ejecutar una prueba básica, cargar un mini grafo de práctica y realizar consultas para analizar relaciones complejas.

Parte 1 — Instalación de Neo4j Desktop

Paso 1 — Descargar

Ingresa al sitio oficial de descarga de Neo4j Desktop

<https://neo4j.com/download>



Seleccionar la versión correspondiente al sistema operativo (Windows / Mac / Linux) y descargar el instalador.

Paso 2 — Instalar

Ejecutar el archivo descargado y completar la instalación como cualquier aplicación estándar.

Paso 3 — Abrir Neo4j Desktop

Al abrir por primera vez, el sistema solicitará iniciar sesión.

Crear una cuenta gratuita si no se posee una.

Parte 2 — Crear la base de datos

1. Hacer clic en **New Project**
2. Asignar nombre al proyecto:
Clase_Grafos
3. Seleccionar **Add → Local DBMS**

4. Asignar nombre a la base:
BD_Grafos
5. Definir una contraseña (guardarla)
6. Presionar **Create**
7. Presionar **Start** para iniciar la base

Cuando el estado indique **Running**, la base está lista.

Parte 3 — Abrir el entorno de consultas

1. Presionar **Open → Neo4j Browser**
2. Se abrirá la consola donde se ejecutarán las consultas Cypher

Parte 4 — Prueba inicial

Ejecutar la siguiente consulta para verificar que la instalación funciona:

```
CREATE (p:Persona {nombre:'Ana'})
RETURN p
```

Si aparece el nodo en pantalla, la base está funcionando correctamente.

Parte 5 — Carga inicial del mini grafo de práctica

Ejecutar el siguiente script completo:

```
CREATE
(alice:Persona {nombre: 'Alice', dni: '30111222'}),
(bruno:Persona {nombre: 'Bruno', dni: '28999111'}),
(acme:Empresa {nombre: 'ACME SRL', rubro: 'Proveedores'}),
(beta:Empresa {nombre: 'Beta SAS', rubro: 'Servicios'}),
(c1:Cuenta {cbu: '00112233'}),
(d1:Domicilio {valor: 'Av. Rivadavia 1234'}),
(dev1:Dispositivo {id: 'DEV-900'}),

(alice)-[:SOCIO_DE {desde: 2023}]->(acme),
(bruno)-[:SOCIO_DE {desde: 2024}]->(beta),
(acme)-[:USA_CUENTA]->(c1),
(beta)-[:USA_CUENTA]->(c1),
(acme)-[:OPERA_DESDE]->(d1),
(beta)-[:OPERA_DESDE]->(d1),
```

(alice)-[:USA_DISPOSITIVO]->(dev1),
(bruno)-[:USA_DISPOSITIVO]->(dev1)

Esto creará un pequeño grafo con personas, empresas, cuentas, domicilios y dispositivos.

Parte 6 — Modificaciones y agregados

Agregar una nueva persona:

```
CREATE (laura:Persona {nombre:'Laura', dni:'33444555'})
```

Relacionarla con una empresa existente:

```
MATCH (l:Persona {nombre:'Laura'}), (e:Empresa {nombre:'ACME SRL'})  
CREATE (l)-[:SOCIO_DE {desde:2025}]->(e)
```

Agregar un nuevo dispositivo:

```
CREATE (d:Dispositivo {id:'DEV-901'})
```

Relacionarlo con Laura:

```
MATCH (l:Persona {nombre:'Laura'}), (d:Dispositivo {id:'DEV-901'})  
CREATE (l)-[:USA_DISPOSITIVO]->(d)
```

Parte 7 — Borrado

Eliminar una persona (y sus relaciones):

```
MATCH (p:Persona {nombre:'Laura'})  
DETACH DELETE p
```

Parte 8 — Consultas

a) Ver todas las personas

```
MATCH (p:Persona)  
RETURN p.nombre, p.dni
```

b) Ver qué empresas tiene cada persona

MATCH (p:Persona)-[:SOCIO_DE]->(e:Empresa)
RETURN p.nombre, e.nombre

c) Detectar empresas que comparten una misma cuenta

MATCH (e:Empresa)-[:USA_CUENTA]->(c:Cuenta)<-[:USA_CUENTA]-(otra:Empresa)
WHERE e <> otra
RETURN e.nombre AS empresa1, otra.nombre AS empresa2, c.cbu

d) Detectar personas que comparten un mismo dispositivo

MATCH (p1:Persona)-[:USA_DISPOSITIVO]->(d:Dispositivo)<-[:USA_DISPOSITIVO]-(p2:Persona)
WHERE p1 <> p2
RETURN p1.nombre, p2.nombre, d.id

e) Detectar empresas que comparten domicilio

MATCH (e1:Empresa)-[:OPERA_DESDE]->(d:Domicilio)<-[:OPERA_DESDE]-(e2:Empresa)
WHERE e1 <> e2
RETURN e1.nombre, e2.nombre, d.valor

f) Buscar una red más completa de sospecha

MATCH (p1:Persona)-[:SOCIO_DE]->(e1:Empresa)-[:USA_CUENTA]->(c:Cuenta)<-[:USA_CUENTA]-(e2:Empresa)<-[:SOCIO_DE]-(p2:Persona)
WHERE p1 <> p2
RETURN p1.nombre, e1.nombre, c.cbu, e2.nombre, p2.nombre

Esta última consulta no analiza un vínculo aislado sino una pequeña red, mostrando cómo dos personas se conectan indirectamente a través de empresas y cuentas compartidas. Aquí se observa el verdadero poder del modelo de grafos para detectar relaciones complejas.